CSCI 6461 | WEEK 1

Introduction to Computing Systems & Binary Foundations

Abstraction, representation, and quantitative performance

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

What Computer Architecture Studies

- Architecture is not just "parts of a computer." It is the study of the **interface contract between software behavior and hardware implementation**.
- The central question: **What must hardware guarantee to software, and how can hardware deliver that guarantee with maximum efficiency?**
- Covers instruction set design, execution pipelines, memory hierarchies, and quantitative trade-offs.
- A functional machine is not necessarily a good machine; architecture evaluates performance, energy, silicon area, and scalability.

Philosophy vs. Reference Platform

- **Philosophy: RISC** (Reduced Instruction Set Computer) principles guide our design analysis.
- **Reference Platform: ARM AArch64** serves as our concrete, production 64-bit architecture throughout the semester.

The Core Abstraction Problem

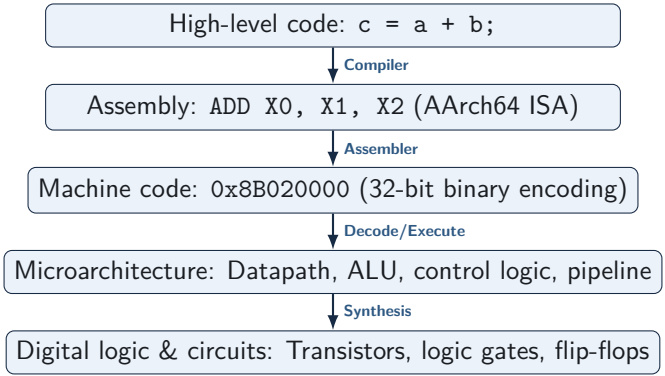
- Programmers write high-level algorithms; hardware manipulates physical voltages and currents.
- Abstraction layers make complex systems manageable.
- Each layer hides implementation details, but hidden details directly determine execution speed and efficiency.

When Abstractions Leak

Abstractions simplify software development, but architecture explains why they leak:

- Memory hierarchy latency
- Integer overflow & width limits
- Cache line collisions
- Pipeline hazards & stalls

System Layers: From Program to Gates



A Concrete Example Across the Layers

High-Level Intent: `c = a + b;`

AArch64 Assembly (RISC Style):

- Operands loaded into 64-bit registers:

```
ADD X0, X1, X2
```

- **Load-Store Model:** ALU instructions operate *only* on registers, never directly on memory.
- **Three-Operand Format:** Preserves source registers (X_1, X_2) non-destructively.

Machine Encoding & Execution:

- Encoded as a fixed 32-bit word: 0x8B020000
- Regular bit fields decode into opcode, destination register (X_0), and source registers (X_1, X_2).
- ALU computes sum; result is latched to register file in 1 clock cycle.

The RISC Philosophy

Core RISC Tenets

- **Fixed-length instructions:** Typically 32 bits, making parallel instruction decode simple and fast.
- **Load-Store architecture:** Only explicit load/store instructions access memory.
- **Large register file:** Reduces memory traffic by keeping active data in hardware registers.
- **Simple addressing modes:** Simplifies datapath design and cycle timing.

Why AArch64?

- ARMv8-A (AArch64) is a clean-slate 64-bit RISC design.
- 31 general-purpose 64-bit registers (`X0`–`X30`) plus zero/stack registers.
- Standardized across mobile, client (Apple Silicon), cloud (AWS Graviton, Ampere), and supercomputers.

ISA vs. Microarchitecture

ISA (The Interface Contract)

- What the software writer / compiler sees.
- Instruction set and encodings.
- Register set (`X0`–`X30`, `SP`, `PC`).
- Memory addressing semantics.
- Exception and privilege models.

Microarchitecture (The Implementation)

- How the hardware delivers the contract.
- In-order vs. out-of-order execution.
- Branch predictor designs & issue widths.
- Cache hierarchy (L1, L2, L3 sizes/latencies).
- Number of physical ALUs and FPUs.

Key Insight: An Apple M-series core and an AWS Graviton core execute identical AArch64 instructions, but use vastly different microarchitectures to optimize for different power/performance budgets.

The Stored-Program Model (von Neumann)

- Instructions and program data share the same memory space as bit patterns.
- A **Program Counter (PC)** tracks the memory address of the next instruction.
- The processor continuously executes the fundamental hardware loop:

$$\textbf{Fetch} \longrightarrow \textbf{Decode} \longrightarrow \textbf{Execute} \longrightarrow \textbf{Memory/Writeback}$$

- Control flow (branches, procedure calls, exceptions) modifies the PC sequence.

Architectural Consequence

Because instructions reside in memory, memory bandwidth and latency dictate instruction delivery rates and overall execution speed.

Core Hardware Components & Memory Hierarchy

- **Registers:** Sub-nanosecond, direct ALU operand access (31 registers in AArch64).
- **Caches (L1–L3):** Fast on-die SRAM buffers.
- **Main Memory (DRAM):** High capacity, order-of-magnitude higher latency.
- **Non-Volatile Storage (SSD/NVMe):** Persistent, orders-of-magnitude slower.

The Memory Wall

CPU compute cycles are fast (~ 0.3–0.5 ns); DRAM accesses are slow (~ 50–100 ns). Architecture focuses heavily on hiding memory latency using caches and pipelines.

Why Binary Is Sufficient

- Digital circuits reliably distinguish two voltage ranges, represented as `0` and `1`.
- An n-bit register generates 2^n distinct bit combinations.
- The exact same 32-bit pattern can represent an unsigned integer, a signed integer, an IEEE 754 float, or an instruction:

Interpretation	**Meaning of** `0x8B020000`
AArch64 Instruction	`ADD X0, X1, X2`
32-bit Signed Integer	$-1,962,803,200$
32-bit Unsigned Integer	$2,332,164,096$
IEEE 754 Single-Precision Float	-1.084×10^{-32}

Core Principle

Hardware stores bit patterns; the executing instruction determines interpretation.

Unsigned Binary Representation

Bit Index (i)	5	4	3	2	1	0
Positional Weight (2^i)	32	16	8	4	2	1
Bit Value (b_i)	1	0	1	1	0	1

- Positional base-2 system:

$$\text{Value} = \sum_{i=0}^{n-1} b_i \cdot 2^i$$

- For 101101_2:

$$1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 45_{10}$$

Fixed-Width Unsigned Ranges

- For an n-bit unsigned number, the range is:

$$[0,\ 2^n - 1]$$

- **8-bit byte:** 0 to 255 $(2^8 - 1)$
- **16-bit halfword:** 0 to $65{,}535$ $(2^{16} - 1)$
- **32-bit word:** 0 to $4{,}294{,}967{,}295$ $(2^{32} - 1 \approx 4.29 \times 10^9)$
- **64-bit doubleword:** 0 to $2^{64} - 1 \approx 1.84 \times 10^{19}$

Architecture Connection

Fixed register widths and immediate bit fields determine the maximum memory addresses and direct constant values that instructions can encode.

Converting Decimal to Binary

Example: Convert 45_{10} to Binary

Decompose into largest powers of 2:

$$45 = 32 + 8 + 4 + 1 = 2^5 + 2^3 + 2^2 + 2^0$$

Weight	32	16	8	4	2	1
Included?	1	0	1	1	0	1

$$45_{10} = 101101_2 = 00101101_2 \text{ (in 8 bits)}$$

Converting Binary to Decimal

Example: Evaluate 110101_2

$$\text{Sum} = \sum b_i \cdot 2^i$$

Weight	32	16	8	4	2	1
Bit	1	1	0	1	0	1

$$32 + 16 + 0 + 4 + 0 + 1 = 53_{10}$$

$$110101_2 = 53_{10}$$

Hexadecimal Representation

- Binary is difficult to read and write without error.
- **Hexadecimal (Base 16)** groups bits into 4-bit chunks (nibbles):

$$16 = 2^4 \implies 1 \text{ hex digit} = 4 \text{ bits}$$

- Two hex digits represent exactly 1 byte (8 bits).
- Eight hex digits represent a 32-bit word; sixteen represent a 64-bit doubleword.
- Memory addresses, machine instructions, and register dumps are formatted in hex.

Binary $\leftrightarrow$ Hex Conversion

Bin	Hex	Bin	Hex	Bin	Hex	Bin	Hex
0000	0	0100	4	1000	8	1100	C
0001	1	0101	5	1001	9	1101	D
0010	2	0110	6	1010	A	1110	E
0011	3	0111	7	1011	B	1111	F

Example: 32-bit AArch64 Instruction Encoding

$$\underbrace{1000}_{\texttt{0x8}}\ \underbrace{1011}_{\texttt{0xB}}\ \underbrace{0000}_{\texttt{0x0}}\ \underbrace{0010}_{\texttt{0x2}}\ \underbrace{0000}_{\texttt{0x0}}\ \underbrace{0000}_{\texttt{0x0}}\ \underbrace{0000}_{\texttt{0x0}}\ \underbrace{0000}_{\texttt{0x0}} \Longrightarrow \texttt{0x8B020000}$$

The Signed-Integer Problem

- Processors must represent negative numbers using only binary bits.
- Desirable properties of a signed representation:
  1. Exactly one representation for zero (no $+0$ and -0).
  2. Standard binary addition hardware handles both addition and subtraction.
  3. Straightforward hardware comparison and overflow detection.
- **Two's Complement** satisfies all these criteria and is universally used in general-purpose computing.

Why Not Sign-Magnitude or Ones' Complement?

Sign-Magnitude

- MSB $=$ Sign, remaining bits $=$ magnitude.
- Yields **two zeros**: $+0$ (0000) and -0 (1000).
- Requires separate adder and subtractor logic circuits.

Ones' Complement

- Negation: invert all bits.
- Still suffers from **dual zeros** (0000 and 1111).
- Requires end-around carry logic during addition.

Solution: Two's complement eliminates duplicate zero and lets a standard adder perform subtraction directly.

Two's-Complement Formulation & Range

- In an n-bit two's complement system, the MSB has a **negative weight**: -2^{n-1}.

$$\text{Value} = -b_{n-1} \cdot 2^{n-1} + \sum_{i=0}^{n-2} b_i \cdot 2^i$$

- Range is asymmetric:

$$\left[-2^{n-1},\ 2^{n-1} - 1\right]$$

- **8-bit signed:** -128 to $+127$
- **32-bit signed:** $-2{,}147{,}483{,}648$ to $+2{,}147{,}483{,}647$
- **64-bit signed:** -2^{63} to $2^{63} - 1$

Negating a Two's Complement Number

Negation Algorithm: $-x = \sim x + 1$

1. Invert all bits (bitwise NOT: $1 \leftrightarrow 0$).
2. Add 1 to the result.

$+5$ in 8-bit binary:	0000 0101	$(+4 + 1)$
Bitwise inversion ($\sim$):	1111 1010	
Add 1 ($+1$):	1111 1011	(represents -5)

- **Bit width matters:** Negation must be performed within the specified register width.

Decoding Negative Bit Patterns

Two Methods to Evaluate `1111 0110` (8-bit signed)

Method 1: Invert and Add 1

$$\texttt{1111 0110} \xrightarrow{\text{invert}} \texttt{0000 1001} \xrightarrow{+1} \texttt{0000 1010} = 10_{10} \implies \mathbf{-10}$$

Method 2: Negative Weight Evaluation (Direct)

$$\text{MSB weight} = -2^7 = -128$$

$$\text{Value} = -128 + 64 + 32 + 16 + 0 + 4 + 2 + 0 = \mathbf{-10}$$

- Direct positional weights avoid manual two-step conversions.

Same Bits, Different Interpretation

8-Bit Hex / Binary	Unsigned Interpretation	Two's Complement
0x00 (0000 0000)	0	0
0x7F (0111 1111)	127	+127
0x80 (1000 0000)	128	−128
0xFE (1111 1110)	254	−2
0xFF (1111 1111)	255	−1

- The hardware adder executes bit operations identically.
- Conditional branch instructions (e.g., `B.LT` signed vs. `B.LO` unsigned in AArch64) determine how status flags are interpreted.

Hardware Subtraction via Addition

Subtraction as Addition

$$A - B = A + (-B) = A + (\sim B + 1)$$

- To subtract:
  1. Invert input bits of B.
  2. Set Carry-In (C_{in}) of the ALU adder to 1.
- A single adder circuit computes both addition and subtraction.

Example: $7 - 5 = 2$ (4-bit)

$$\begin{array}{rl} 0111 & (+7) \\ +\ 1011 & (-5) \\ \hline (1)\ 0010 & (+2) \end{array}$$

Carry-out bit is discarded for signed arithmetic.

Carry vs. Signed Overflow

Unsigned Carry (C Flag)

- Generated by MSB carry-out during addition.
- Indicates unsigned result $> 2^n - 1$.

Signed Overflow (V Flag)

- Result sign is mathematically invalid for signed inputs.
- $(+) + (+) = (-)$ or $(-) + (-) = (+)$.

4-Bit Signed Overflow Example ($+7 + +3$)

$$0111_2\ (+7) + 0011_2\ (+3) = 1010_2\ (-6 \text{ in 4-bit two's complement})$$

Because the range is $[-8, +7]$, $+10$ overflows into the sign bit. Hardware asserts $V = 1$.

Sign Extension vs. Zero Extension

Sign Extension (Signed)

- Replicate the MSB into all new higher-order bit positions.
- Preserves negative signed value.

$$\texttt{1111 1011}_2\ (-5_{10})$$

$$\downarrow$$

$$\texttt{1111 1111 1111 1011}_2\ (-5_{10})$$

Zero Extension (Unsigned)

- Fill all new higher-order bits with 0.
- Preserves unsigned magnitude.

$$\texttt{1111 1011}_2\ (251_{10})$$

$$\downarrow$$

$$\texttt{0000 0000 1111 1011}_2\ (251_{10})$$

Common Representation Bugs in Systems

- **Signed vs. Unsigned Comparisons:** Branching on unsigned comparison when evaluating negative signed integers (e.g., $-1 > 0$ if evaluated as $2^{32} - 1$).
- **Truncation Bugs:** Casting a 64-bit address or large integer into a 32-bit register discards high-order bits without error notifications.
- **Security Vulnerabilities:** Buffer allocations where arithmetic overflow causes allocation of small memory buffers followed by large memory copies.
- **Sign Extension Mistakes:** Loading an 8-bit signed byte into a 64-bit register with zero-extension produces large positive values instead of negative integers.

Defining CPU Performance

- **Wall-clock (Elapsed) Time:** Total time from start to completion, including disk I/O, OS scheduling, and waiting for memory.
- **CPU Execution Time:** Actual time the processor spends computing on behalf of the specific program:

$$T_{\text{CPU}} = \text{User CPU Time} + \text{System CPU Time}$$

- **Clock Frequency (f_{clk}):** Cycles per second (Hz).
- **Clock Period (T_{clk}):** Duration of a single clock cycle:

$$T_{\text{clk}} = \frac{1}{f_{\text{clk}}}$$

Latency vs. Throughput

Latency (Execution Time)

- Time elapsed to complete one discrete task.
- Unit: seconds, milliseconds, clock cycles.
- Critical for real-time systems, UI responsiveness, and single-thread execution.

Throughput (Bandwidth)

- Total work completed per unit time.
- Unit: tasks/sec, instructions/cycle (IPC), GB/s.
- Critical for servers, data centers, and parallel throughput.

Takeaway: Pipelining and multicore architectures increase throughput, often while leaving individual instruction latency unchanged or slightly higher.

Clock Cycles and Dynamic Execution

- Program execution time can be expressed in terms of processor clock cycles:

$$T_{\text{CPU}} = \text{Clock Cycles for Program} \times T_{\text{clk}} = \frac{\text{Clock Cycles for Program}}{f_{\text{clk}}}$$

- **Dynamic Instruction Count (IC):** The actual number of instructions executed at runtime (distinct from static program size in binary).
- Loops, recursive calls, and input-dependent branches determine IC.

Cycles Per Instruction (CPI)

- **CPI:** Average clock cycles required per executed instruction across a dynamic workload:

$$CPI = \frac{\text{Clock Cycles}}{IC}$$

- Alternatively, **IPC (Instructions Per Cycle)** $= 1/CPI$.
- Different instruction classes require different cycle counts:

$$\text{Clock Cycles} = \sum_{i=1}^{k} (IC_i \times CPI_i)$$

- Memory loads and stalls increase effective CPI; superscalar execution brings average CPI below 1.

The Iron Law of Processor Performance

The Iron Law Formulation

$$T_{\text{CPU}} = IC \times CPI \times T_{\text{clk}} = \frac{IC \times CPI}{f_{\text{clk}}}$$

Factor	Units	Primary Determining Factors
IC	$\frac{\text{Instructions}}{\text{Program}}$	Algorithm, Compiler, ISA Architecture
CPI	$\frac{\text{Cycles}}{\text{Instruction}}$	Microarchitecture, Memory Hierarchy, Pipeline
T_{clk}	$\frac{\text{Seconds}}{\text{Cycle}}$	Circuit Design, Semiconductor Process Node

Iron Law: Worked Example

Machine A

- $IC = 1.0 \times 10^9$
- $CPI = 2.0$
- $f_{\text{clk}} = 2.0$ GHz ($T_{\text{clk}} = 0.5$ ns)

$$T_{\text{CPU}} = \frac{1.0 \times 10^9 \times 2.0}{2.0 \times 10^9 \text{ s}^{-1}} = \mathbf{1.0} \text{ s}$$

Machine B

- $IC = 1.2 \times 10^9$ (More instructions)
- $CPI = 1.2$ (Better pipeline)
- $f_{\text{clk}} = 2.4$ GHz ($T_{\text{clk}} \approx 0.417$ ns)

$$T_{\text{CPU}} = \frac{1.2 \times 10^9 \times 1.2}{2.4 \times 10^9 \text{ s}^{-1}} = \mathbf{0.6} \text{ s}$$

Machine B completes the workload $1.67\times$ **faster** despite executing 20% more instructions.

Why Clock Frequency Alone Is Misleading

- The "Megahertz Myth": Clock speed alone does not equal performance.
- A 4 GHz processor with $CPI = 2.5$ is slower than a 3 GHz processor with $CPI = 1.0$ on the same instruction stream:

$$\text{Proc 1:} \quad \frac{IC \times 2.5}{4.0 \times 10^9} = IC \times 0.625 \text{ ns}$$

$$\text{Proc 2:} \quad \frac{IC \times 1.0}{3.0 \times 10^9} = IC \times 0.333 \text{ ns} \quad (\approx 1.88\times \text{ faster!})$$

- Hardware designs balancing high frequency against deep, stall-prone pipelines often lose in total execution time.

Quantifying Speedup

Definition of Speedup

$$\text{Speedup} = \frac{T_{\text{old}}}{T_{\text{new}}} = \frac{\text{Performance}_{\text{new}}}{\text{Performance}_{\text{old}}}$$

- If execution time decreases from 10 s to 5 s:

$$\text{Speedup} = \frac{10}{5} = 2.0\times$$

- If execution time decreases from 10 s to 8 s:

$$\text{Speedup} = \frac{10}{8} = 1.25\times$$

- Always define the reference workload and baseline environment.

Week 1 Synthesis and Next Steps

- **ISAs** define the boundary between machine code and hardware execution.
- **RISC Philosophy** prioritizes simple, uniform instructions and register-to-register operations to enable streamlined microarchitecture.
- **Representations** (binary, two's complement, hex) determine the range, precision, and simplicity of underlying ALU circuits.
- **The Iron Law** provides the quantitative foundation to reason about execution time trade-offs.
- **Next Week:** The AArch64 Programmer's Model, General-Purpose Registers (`X0`–`X30`), and Core Instructions (`ADD`, `SUB`, `LDR`, `STR`).